FactorMiner: A Self-Evolving Agent with Skills and Experience Memory for Financial Alpha Discovery

Yanlong Wang
Tsinghua Shenzhen International Graduate School, Tsinghua University
Pengcheng Laboratory
Shenzhen, China
wangyanl21@mails.tsinghua.edu.cn

Jian Xu
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
xujian20@mails.tsinghua.edu.cn

Hongkang Zhang
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
zhanghk21@mails.tsinghua.edu.cn

Shao-Lun Huang*
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
shaolun.huang@sz.tsinghua.edu.cn

Danny Dongning Sun*
Pengcheng Laboratory
Shenzhen Finance Institute, Chinese University of Hong Kong
Shenzhen, China
ds316@columbia.edu

Xiao-Ping Zhang*
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
xpzhang@ieee.org

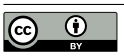
Abstract

Formulaic alpha factor mining is a critical yet challenging task in quantitative investment, characterized by a vast search space and the need for domain-informed, interpretable signals. However, finding novel signals becomes increasingly difficult as the library grows due to high redundancy. We propose **FactorMiner**, a lightweight and flexible self-evolving agent framework designed to navigate this complex landscape through continuous knowledge accumulation. FactorMiner combines a **Modular Skill Architecture** that encapsulates systematic financial evaluation into executable tools with a structured **Experience Memory** that distills historical mining trials into actionable insights (successful patterns and failure constraints). By instantiating the Ralph Loop paradigm—retrieve, generate, evaluate, and distill—FactorMiner iteratively uses memory priors to guide exploration, reducing redundant search while focusing on promising directions. Experiments on multiple datasets across different assets and markets show that FactorMiner constructs a diverse library of high-quality factors with competitive performance, while maintaining low redundancy among factors as the library scales. Overall, FactorMiner provides a practical approach to scalable discovery of interpretable formulaic alpha factors under the "Correlation Red Sea" constraint.

CCS Concepts

• **Information systems** → **Data mining**; • **Computing methodologies** → **Intelligent agents**; • **Applied computing** → **Economics**.

*Corresponding authors.



KDD '26, Jeju Island, Republic of Korea

ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3818978

Keywords

Quantitative Investment; Formulaic Alpha; Factor Mining Agent Skill; Experience Memory; Parallel Evaluation; Intraday Prediction

ACM Reference Format:
Yanlong Wang, Jian Xu, Hongkang Zhang, Shao-Lun Huang, Danny Dongning Sun, and Xiao-Ping Zhang. 2026. FactorMiner: A Self-Evolving Agent with Skills and Experience Memory for Financial Alpha Discovery. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3818978

1 Introduction

Discovering predictive alpha factors is central to quantitative trading and portfolio construction. In practice, automated factor discovery faces three fundamental challenges: (i) *vast search complexity*—the space of formulaic expressions grows combinatorially with operator compositions and parameters; (ii) *poor knowledge accumulation*—traditional search methods (genetic programming, reinforcement learning) fail to retain and reuse insights across exploration sessions, leading to repetitive trials; (iii) *interpretability constraints*—unlike black-box neural predictors, financial practitioners require transparent, auditable formulas with explicit financial logic for regulatory compliance and risk management.

Traditional approaches rely heavily on domain experts who manually craft factors based on financial intuition [5, 13]. More recently, machine learning methods have been applied to asset pricing [7, 9], demonstrating strong predictive power but often sacrificing interpretability. Genetic programming [16] and reinforcement learning [41, 42] offer automated search but suffer from knowledge forgetting: as the factor library grows and correlation constraints tighten, these methods lack mechanisms to accumulate structural knowledge about what works and what fails. Moreover, existing methods treat each mined factor in isolation without considering the global factor library perspective, they optimize individual factor quality but ignore how new factors interact with the existing library, leading to redundant discoveries and inefficient exploration.

We study formulaic factor mining as a task for self-evolving AI agents [36, 40]. Each factor is an explicit expression over market

fields (e.g., close, volume, VWAP) composed from a library of 60+ operators. Unlike end-to-end neural approaches, formulaic factors enable human-in-the-loop auditing and compositional generalization across market regimes. The key research question is: *how can an agent efficiently explore this vast program space while maintaining a global view of the factor library and autonomously accumulating structural knowledge across exploration sessions?*

We propose **FactorMiner**, a self-evolving agent framework that addresses these challenges through two synergistic mechanisms: a compositional skill architecture and experience memory. First, we design factor mining as a reusable agent skill that can be invoked on-demand by a language model agent [33, 40]. The skill encapsulates domain knowledge—a curated operator library with 60+ financial operators, a multi-stage validation pipeline with IC thresholds and correlation checks, and standardized evaluation protocols—enabling flexible task decomposition and independent upgrades without retraining the agent. Second, we introduce experience memory that enables the agent to self-evolve through accumulated knowledge [28, 36]. The memory stores distilled structural patterns from historical mining sessions: successful patterns (factor templates that consistently pass quality thresholds) and forbidden regions (factor families with high mutual correlation to existing library members). Crucially, this memory maintains a global factor library perspective: the agent decides mining directions by considering how candidate factors complement the existing library, rather than optimizing individual factors in isolation.

The agent adopts the Ralph Loop paradigm for self-evolution: retrieve relevant patterns from experience memory, generate candidate factors by invoking the mining skill with retrieved priors, evaluate candidates through parallel validation, and distill outcomes back into memory. This creates a positive feedback cycle where each mining session improves future exploration efficiency, enabling the agent to continuously refine its search strategy.

To ensure computational efficiency, we build a lightweight yet high-performance system leveraging GPU-accelerated operators, multi-process parallelization, and C-compiled efficient numerical operations. This yields substantial speedups over standard Python implementations (e.g., NumPy/Pandas), making large-scale iterative evaluation computationally feasible. We summarize our main contributions as follows:

- **Experience memory for agent-based factor mining.** We introduce structured experience memory into agent-based factor discovery, enabling self-evolution through accumulated knowledge. The memory stores reusable patterns that guide exploration and reduce redundant search compared to memoryless baselines.
- **Modular skill architecture for on-demand invocation.** We design factor mining as a compositional, reusable skill that encapsulates domain knowledge in a standalone module. This enables flexible agent invocation, independent skill upgrades, and clear separation between agent reasoning and skill execution.
- **Lightweight and efficient mining system.** We build a high-performance factor evaluation engine using GPU accelerated operators, multi-process parallelization, and C-compiled efficient numerical operations, enabling large-scale factor mining. This supports rapid iteration and large-scale factor exploration while adapting to different server configurations and compute budgets.
- **Global factor library perspective.** We incorporate factor library admission mechanisms into the mining loop, enabling the agent to make mining decisions from a global library perspective. The agent considers how candidate factors complement the existing library, rather than optimizing individual factors in isolation.
- **Open factor library with research and practical value.** We provide 110 A-share equity factors with explicit formulaic expressions, validated on real market data. These interpretable factors serve as diagnostic tools to understand cross-market behavioral anomalies and market microstructure inefficiencies, offering both research insights and practical value for quantitative trading.

2 Related Work

2.1 Automated Alpha Factor Discovery

Formulaic alpha libraries provide explicit, human-readable expressions valuable for interpretable research. Kakushadze [13] presents 101 formulaic alphas with explicit expressions and low mutual correlations (15.9%), showing that compact formula libraries can be both diverse and effective; widely used industry variants such as Alpha191 extend this with around 191 expressions covering additional microstructure patterns. The discovery process behind such libraries is often undocumented and may combine expert design with automated search (e.g., genetic programming), but the released artifacts are explicit formulas. These classical libraries remain largely limited to daily-frequency data, and the interpretability of deeply nested expressions can degrade, so high-frequency markets still lack comparable curated factor libraries for systematic research.

To automate factor discovery, evolutionary program search methods such as genetic programming represent factor formulas as executable programs and evolve them via crossover and mutation [2, 16, 23]. In practice, vanilla genetic programming can explore the expression space inefficiently, exhibiting slow convergence and limited semantic guidance because genetic operators primarily act on program syntax rather than program behavior [21, 30]. More recently, machine learning methods have been applied to empirical asset pricing and high-dimensional model selection [7, 9, 11, 12, 17]. While these methods achieve strong predictive performance, they are often less transparent than formulaic signals and can be difficult to interpret or audit in high-stakes settings [32].

Reinforcement learning has been used to navigate the discrete space of formulaic factor expressions by treating evaluation metrics (e.g., IC/ICIR) as rewards [41, 42], often at the cost of additional training and repeated evaluation overhead. In parallel, neural and LLM-driven frameworks generate and refine formulaic factors with exploration strategies designed to mitigate decay and redundancy [35, 38]. Despite these advances, these approaches still face the challenge of explicitly and persistently reusing structural patterns across mining sessions; as the factor library grows and redundancy/correlation constraints tighten, exploration can become increasingly repetitive [7, 17].

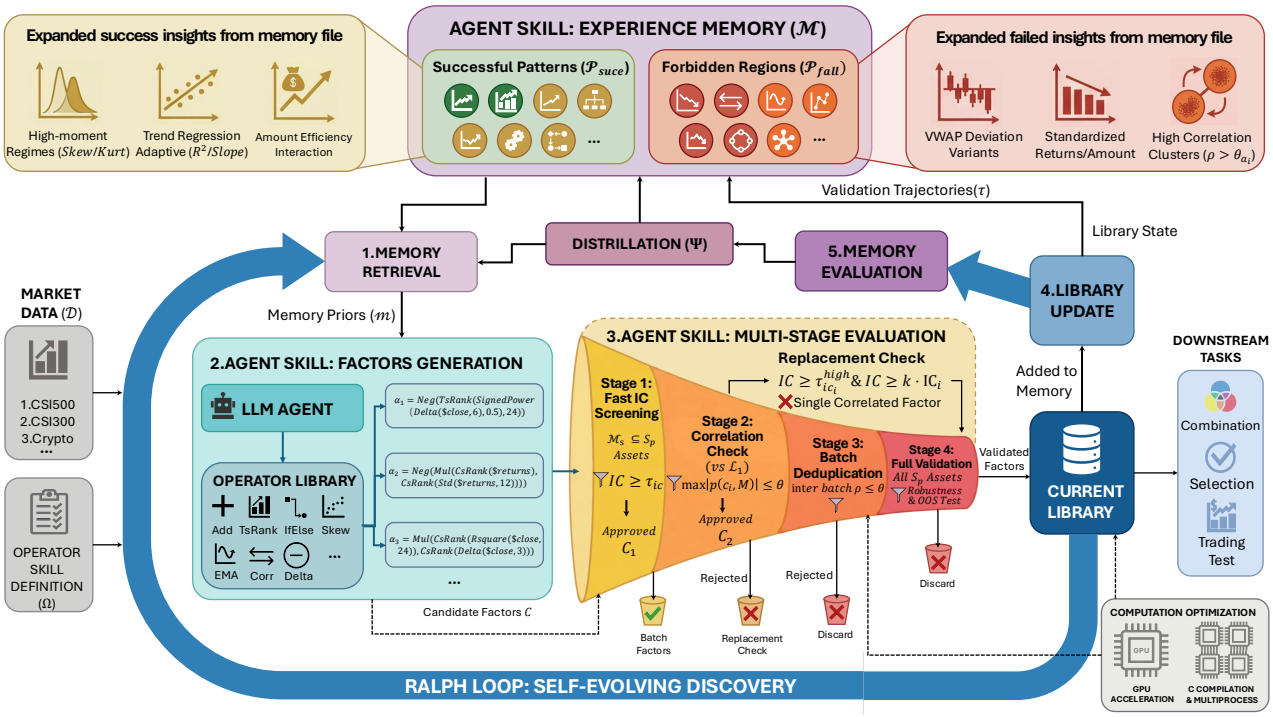


Figure 1: FactorMiner System Architecture. The Ralph Loop framework integrates three key components: (1) Experience Memory that stores successful patterns and forbidden regions from past mining sessions; (2) Agent Skill that encapsulates the multi-stage validation pipeline (IC screening, correlation checking, deduplication, and full validation); (3) Factor Library that grows dynamically while maintaining orthogonality constraints. The agent iteratively retrieves memory priors, generates candidates through the skill, and distills outcomes back into memory for improved future exploration.

2.2 AI Agents with Skills and Memory

Recent language-model agents shift from one-shot text generation to closed-loop task execution via tool calls and feedback. Toolformer [33] trains LLMs to insert API calls in a self-supervised manner, while ReAct [40] interleaves reasoning traces with actions—the agent plans, invokes tools, observes outcomes, and iterates. This tool-augmented, skill-based view motivates our design: factor mining is packaged as an executable skill that an agent can invoke on demand. Packaging domain procedures as reusable skills lets an agent tackle complex, specialized tasks without fine-tuning model weights, while cutting repeated in-context instructions and their token overhead and improving reliability on multi-step domain workflows.

Memory and self-improvement mechanisms further let agents accumulate experience over time. Reflexion [36] uses language-based self-reflection to store lessons in natural language, and generative agents [28] maintain memory streams (e.g., observations, reflections, plans) to condition future behavior; more broadly, continual learning studies how to retain knowledge without catastrophic forgetting [27]. In our setting, we instantiate memory for symbolic program synthesis: instead of storing dense representations or replay buffers [4], we retain reusable symbolic rules and structural patterns discovered during factor mining, together with summary statistics that help avoid redundant regions of the search space.

From a theoretical perspective, meta-learning and meta-RL formalize "learning to learn" by extracting transferable learning biases from prior tasks or episodes [8, 10, 24]. Analogously, a memory-guided mining process can be seen as acquiring search priors: beyond producing individual factors, the system accumulates structural knowledge that shapes future exploration.

Beyond learning to call tools, systems work has advocated modular agent architectures that route subproblems to specialized models, external knowledge sources, and discrete reasoning modules (e.g., MRKL [14]; HuggingGPT [34]). In parallel, recent benchmarks and training pipelines study how to ground LLM outputs into executable API calls at scale and reduce tool hallucination, including API-Bank [18], ToolLLM/ToolBench [31], and Gorilla/APIBench [29]. These works collectively emphasize the importance of separating high-level planning from reliable execution, and of representing tools/skills in a form that supports retrieval and continual updates.

Another line of research addresses long-horizon agent behavior under limited context windows by introducing explicit long-term memory managers. MemGPT [26] proposes OS-inspired hierarchical memory with controlled paging between fast and slow memory, while Voyager [39] demonstrates an open-ended embodied agent that accumulates an executable skill library and reuses it to generalize to new tasks. Such settings are often effectively non-stationary: as an agent acquires new tools/skills (or as an external library grows), the feasible action space and redundancy patterns evolve,

motivating experience summarization mechanisms that capture reusable patterns and avoid repeatedly exploring known dead ends.

3 Methodology

3.1 Problem Formulation

We define the alpha factor discovery task over a universe of M assets and a time horizon T. The raw market input is represented by a tensor $\mathcal{D} \in \mathbb{R}^{M\times T\times F}$, where each entry $d_{m,t}^{(j)}$ denotes the value of feature j (e.g., price, volume) for asset m at time t. **Symbolic factor space.** A symbolic alpha factor α is a computational program composed from an operator set Ω that transforms market states into a cross-sectional predictive signal $\mathbf{s}_t \in \mathbb{R}^M$:

$$\alpha : \mathcal{D}_{:,:t,:} \to \mathbf{s}_t, \quad s_t^{(m)} = \alpha(\mathbf{d}_{m,:t}) \tag{1}$$

where $s_t^{(m)}$ represents the predictive score for asset m relative to its peers. Each operator $o \in \Omega$ has a fixed arity and a typed signature (e.g., time-series $\to$ time-series, cross-section $\to$ cross-section), and programs $\alpha \in \mathcal{P}(\Omega)$ correspond to expression trees composed from Ω with admissible parameters; detailed definitions and categories of Ω are provided in Appendix A and Table 7.

The effectiveness of α is quantified by the Information Coefficient (IC), defined as the cross-sectional Spearman rank correlation between the signal $\mathbf{s}_t$ and subsequent returns $\mathbf{r}_{t+1} \in \mathbb{R}^M$:

$$\mathrm{IC}_t(\alpha) = \mathrm{Corr}_{\mathrm{rank}}\big(\mathbf{s}_t, \mathbf{r}_{t+1}\big) \tag{2}$$

Consistency over time is measured by the Information Ratio (ICIR): $\mathrm{ICIR}(\alpha) = \mu(\mathrm{IC}_t)/\sigma(\mathrm{IC}_t)$.

Correlation metric. To enforce library diversity, we measure redundancy between two factors α and β by the time-average cross-sectional Spearman correlation of their realized signals:

$$\rho(\alpha,\beta) = \frac{1}{|\mathcal{T}|}\sum_{t\in\mathcal{T}} \mathrm{Corr}_{\mathrm{rank}}\big(\mathbf{s}_t(\alpha), \mathbf{s}_t(\beta)\big), \tag{3}$$

where $\mathcal{T}$ denotes the set of evaluation timestamps. This definition matches our current implementation, but the framework can accommodate alternative dependence measures (e.g., time-series correlation, partial correlation, or non-linear dependence metrics) by replacing $\rho(\cdot,\cdot)$ accordingly.

Objective: Orthogonal Library Synthesis. FactorMiner treats the problem as the iterative construction of a diverse factor library $\mathcal{L} = \{\alpha_1, \ldots, \alpha_K\}$. The goal is to maximize the aggregate predictive quality of the library subject to a global redundancy constraint:

$$\mathcal{L}^* = \arg\max_{\mathcal{L}\subset\mathcal{P}} \sum_{\alpha\in\mathcal{L}} \Phi(\alpha) \quad \text{s.t.} \quad \forall \alpha_i \neq \alpha_j \in \mathcal{L} : |\rho(\alpha_i,\alpha_j)| < \theta \tag{4}$$

where $\mathcal{P}$ is the infinite space of constructible programs, $\Phi(\cdot)$ is a fitness metric, and θ is the correlation budget.

The Correlation Red Sea. As the library $\mathcal{L}$ populates, the feasible region for new orthogonal factors—$\mathcal{P}_{\mathrm{orth}} = \{\alpha \in \mathcal{P} : \max_{g\in\mathcal{L}} |\rho(\alpha, g)| < \theta\}$—shrinks rapidly as diversity constraints tighten. Standard search methods (e.g., GP or RL) often get trapped in this correlation red sea because they lack mechanisms to track explored regions or failure patterns.

Decision-Theoretic Memory Formulation. To navigate the correlation red sea, we reformulate the discovery process as a sequential decision task over an evolving internal knowledge state $S_t = (\mathcal{L}_t, \mathcal{M}_t)$, where $\mathcal{M}_t$ represents the persistent experience memory. The agent first retrieves a context-dependent memory signal m_t from $\mathcal{M}_t$ and $\mathcal{L}_t$, and then samples candidates from a memory-conditioned policy:

$$\alpha \sim \pi(\alpha \mid m_t), \quad m_t = R(\mathcal{M}_t, \mathcal{L}_t) \tag{5}$$

The role of $\mathcal{M}$ is to induce a probability measure contraction over the program space $\mathcal{P}$. By distilling historical trajectories $\tau = \{(\alpha_i, R_i)\}_{i=1}^B$ into structured patterns, $\mathcal{M}$ shifts the sampling mass toward the orthogonal manifold $\mathcal{P}_{\mathrm{orth}}$. Mathematically, the evolution of memory is governed by a distillation operator Ψ:

$$\mathcal{M}_{t+1} = \Psi(\mathcal{M}_t, \tau_t) \tag{6}$$

which updates the agent's belief distribution so that future exploration is steered toward higher-utility and lower-redundancy regions of $\mathcal{P}$.

3.2 Factor Mining Skill Architecture

Unlike traditional RL or monolithic agent approaches where domain logic is often hard-coded or entangled with the agent's reasoning loop, FactorMiner adopts a modular skill-based architecture. Inspired by the tool-augmented paradigm [33], we encapsulate the entire factor mining process as a standalone, reusable Agent Skill—a standardized interface that exposes high-level capabilities to the LLM while abstracting away low-level execution details. The overall interaction pattern is illustrated in Figure 1.

Compositional Design. The skill is structured as a hierarchical library of tools:

- **Operator Layer**: A curated set of 60+ financial operators (e.g., `TsRank`, `Rsquare`) implemented with GPU-accelerated backends. This ensures that the agent's symbolic proposals are executable and computationally efficient.
- **Validation Pipeline**: A rigorous, standardized protocol for factor assessment (`check_ic` → `check_correlation` → `admit`). By decoupling validation from generation, we ensure that the agent's "creativity" is bounded by strict quantitative constraints, reducing hallucinated or invalid candidates.

Advantages of the Skill-Based Approach. This modular design offers three distinct advantages over monolithic architectures:

(1) **Prevention of Calculation Hallucination**: Large language models often struggle with precise arithmetic and algorithmic execution. By offloading the evaluation to a deterministic, code-based skill, FactorMiner avoids "hallucinated metrics": the reported IC and ICIR are computed deterministically.
(2) **Cross-Domain Transferability**: The skill is parameterized to support multiple markets (e.g., A-shares vs. Crypto) via configuration files. The same high-level agent reasoning logic can be applied to different financial domains simply by switching the underlying skill context, as demonstrated in our cross-market experiments (Section 4.2.2).
(3) **Independent Optimization**: The skill's execution backends and evaluation pipeline can be optimized independently of the agent's reasoning model, improving throughput without retraining the LLM backbone.

3.3 Experience Memory

A key component of FactorMiner is the experience memory $\mathcal{M}$, a structured knowledge base that accumulates insights from historical mining sessions. We formalize the dynamics of the memory system through three conceptual operators: Formation, Evolution, and Retrieval.

Memory Formation. At the end of each mining batch t, the agent analyzes the mining trajectory $\tau_t = \{(\alpha_i, R_i)\}_{i=1}^{B}$, where R_i represents the evaluation feedback (IC, correlation, etc.). A formation operator F selectively extracts informational artifacts:

$$\mathcal{M}_{t+1}^{\text{form}} = F(\mathcal{M}_t, \tau_t) \tag{7}$$

This process distills raw data into symbolic patterns, categorizing them into **successful patterns** $\mathcal{P}_{\text{succ}}$ (those that pass admission) and **forbidden regions** $\mathcal{P}_{\text{fail}}$ (those rejected due to high correlation).

Memory Evolution. Formed memory candidates are integrated into the existing knowledge base through an evolution operator E:

$$\mathcal{M}_{t+1} = E(\mathcal{M}_t, \mathcal{M}_{t+1}^{\text{form}}) \tag{8}$$

This operator consolidates redundant entries and discards low-utility information. For instance, if a specific `VWAP_Deviation` variant is admitted but shows 0.82 correlation with existing factors, it is reclassified into $\mathcal{P}_{\text{fail}}$ to prevent further redundant exploration.

Memory Retrieval. During the factor generation phase, the agent retrieves a context-dependent memory signal m_t via a retrieval operator R:

$$m_t = R(\mathcal{M}_t, \mathcal{L}_t) \tag{9}$$

The signal m_t serves as a prompt-level constraint for the LLM policy, effectively shaping the sampling distribution $\pi(\alpha \mid m_t)$. In practice, we store experience as compact natural-language templates with canonical examples (e.g., "recommended directions" and "forbidden directions"), and retrieve them by matching against the current library diagnostics and recent rejection reasons.

Memory Content. The resulting memory $\mathcal{M}$ maintains a persistent record of the evolving mining landscape:

(1) Mining State ($\mathcal{S}$): Tracks the global evolution of the factor library, including current library size $|\mathcal{L}|$, recent admission logs, and saturation metrics that signal when specific logical domains (e.g., price-volume reversal) are becoming overpopulated.

(2) Structural Experience ($\mathcal{P}$): This is the core of the agent's guidance system, categorized into:

- **Recommended Directions ($\mathcal{P}_{\text{succ}}$):** High-success logical templates distilled from recent batches, such as higher moment regimes (using Skew/Kurt for environment switching) and robust efficiency interaction.
- **Forbidden Directions ($\mathcal{P}_{\text{fail}}$):** Regions identified as "Red Seas" due to persistent high correlation with the existing library, such as simple VWAP Deviations or standardized returns.

(3) Strategic Insights ($\mathcal{I}$): High-level lessons learned from the mining process, such as the observation that non-linear combination strategies (e.g., XGBoost-based synthesis) tend to outperform linear ones, or specific operator warnings (e.g., the instability of high-order moments in high-frequency data).

3.4 Ralph Loop: Self-Evolving Factor Discovery

FactorMiner adopts the **Ralph Loop** paradigm—an iterative refinement philosophy where agents accumulate experience and self-evolve through repeated interaction. We instantiate this paradigm for factor mining by integrating experience memory into the search process (Algorithm 1). The key insight is that memory enables the agent to learn how to search—avoiding redundant exploration while focusing on promising regions.

Algorithm 1 Ralph Loop: Self-Evolving Factor Discovery

Input: Operator library Ω, experience memory $\mathcal{M}$, target library size K
Output: Factor library $\mathcal{L}$
Initialize $\mathcal{L} \leftarrow \emptyset$
repeat
  Step 1: Memory Retrieval
    Retrieve memory signal $m \leftarrow R(\mathcal{M}, \mathcal{L})$
  Step 2: Guided Generation
    Sample batch $C \sim \pi(\alpha \mid m)$ using Ω
  Step 3: Multi-Stage Evaluation
    Stage 1: Fast IC screening (M_{fast} assets)
      $C_1 \leftarrow \{\alpha \in C : |\text{IC}(\alpha)| \geq \tau_{\text{IC}}\}$
    Stage 2: Correlation check against $\mathcal{L}$
      $C_2 \leftarrow \{\alpha \in C_1 : \max_{g \in \mathcal{L}} |\rho(\alpha, g)| < \theta\}$
    Stage 2.5: Replacement check
      For $\alpha \in C_1 \setminus C_2$, let $g^\star = \arg\max_{g \in \mathcal{L}} |\rho(\alpha, g)|$
      Replace $g^\star$ with α if $|\rho(\alpha, g^\star)| \geq \theta$,
      $\max_{g \in \mathcal{L} \setminus \{g^\star\}} |\rho(\alpha, g)| < \theta$, and $\Phi(\alpha) \geq \Phi(g^\star) + \Delta$
    Stage 3: Batch deduplication (intra-batch $\rho < \theta$)
    Stage 4: Full validation (M_{full} assets) and trajectory τ collection
  Step 4: Library Update
    Admit validated factors: $\mathcal{L} \leftarrow \mathcal{L} \cup C_{\text{admitted}}$
  Step 5: Memory Evolution
    Update strategy: $\mathcal{M} \leftarrow E(\mathcal{M}, F(\tau))$
until $|\mathcal{L}| \geq K$ or budget exhausted

Our instantiation of the Ralph Loop for factor mining has four key properties:

Global library perspective. Unlike methods that optimize individual factors in isolation, our approach considers how each candidate complements the existing library $\mathcal{L}$. The correlation constraint (Stage 2) ensures diversity, while the replacement mechanism (Stage 2.5) allows high-quality factors to replace inferior ones.

Memory-guided exploration. By maintaining $\mathcal{P}_{\text{succ}}$ and $\mathcal{P}_{\text{fail}}$, the agent avoids redundant exploration of known failure regions while focusing on promising structural patterns.

Multi-stage evaluation. The multi-stage pipeline balances efficiency and accuracy: Stage 1 uses a small asset subset for fast screening, Stages 2–3 enforce correlation constraints (inter-factor and intra-batch), and Stage 4 performs full validation only on surviving candidates.

Self-evolution. After each iteration, the memory is updated via the evolution operator E (integrating insights from F), creating a feedback loop where the agent continuously improves its search strategy. This enables continual learning: knowledge accumulated in early sessions benefits later exploration.

Trajectory semantics. The trajectory τ_t records, for each evaluated candidate, its formula α, quality statistics (e.g., IC/ICIR), redundancy diagnostics (e.g., $\max_{g \in \mathcal{L}} |\rho(\alpha, g)|$), and the rejection or admission outcome (including whether a replacement was triggered). These fields are the inputs to the formation operator F and support experience distillation across batches.

Detailed specifications of the operator library and admission criteria are provided in Section A.

To illustrate the diversity of the resulting library, Figure 2 visualizes the correlation structure of the released full A-share factor library (110 admitted factors): most factor pairs are weakly to moderately correlated, with only a few localized clusters of higher dependence.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate FactorMiner across A-share equities and the cryptocurrency market to assess its discovery efficiency and cross-asset generalization. For A-share equities, we utilize intraday 10-minute bars of three index universes: the CSI 500 and CSI 1000 index constituents for mid-cap and broader small-/mid-cap coverage, and the CSI 300 index constituents for large-cap representation, with over 25 million data points in aggregate. For the Cryptocurrency market, we use 10-minute bars of 64 major assets from Binance. All datasets cover a training period from 2024-Q1 to 2024-Q4 and a held-out test period in 2025. The prediction target is instantiated as the non-overlapping next-step return ratio at the same 10-minute frequency, while the mining framework can accommodate alternative targets and horizons.

Baselines and Metrics. We compare against six representative methods: (1) *Alpha101 (Classic)* [13], a static library of hand-crafted formulas; (2) *Alpha101 (Adapted)*, with parameters tuned for high-frequency data; (3) *Random Formula Exploration (RF)*, randomly sampling type-correct expression trees from $\mathcal{P}(\Omega)$ under a bounded depth/size distribution; (4) *GPLearn* [37], an enhanced genetic programming approach; (5) *AlphaForge* [35], a neural framework for mining and dynamically combining formulaic alpha factors; and (6) *AlphaAgent* [38], an LLM-driven proposal-refinement framework. For a fair comparison, we apply the same admission rules to each method and evaluate an equal-sized factor set; if a method admits fewer factors, we complete the set by selecting the remaining candidates with the best IC. We also include a *No Memory* variant as an internal baseline in Section 4.3. Performance is quantified using standard predictive metrics: *Rank Information Coefficient* for forecasting precision, and its ratio (ICIR) for stability across time. Unless otherwise stated, we use IC to denote this Spearman correlation based IC. When applicable, all baselines share the same operator library Ω, data fields, and evaluation/admission protocol, and are scored with a unified evaluation engine. For methods that require LLM-based proposal generation, we use Gemini 3.0 Flash unless otherwise stated. We do not fine-tune or update the LLM weights; self-evolution comes from retrieval-conditioned generation, deterministic skill execution, library admission, and memory distillation.

4.2 Main Results

4.2.1 Factor Quality and Diversity. Table 1 reports 2025 out-of-sample results under a strict protocol: Top-40 factors are selected once on CSI500 (2024) and then frozen for evaluation on multiple datasets. Under this protocol, **FactorMiner** performs competitively among the compared methods across the four markets. In the Factor Library setting, it achieves IC/ICIR of **8.25%**/**0.77** on CSI500,

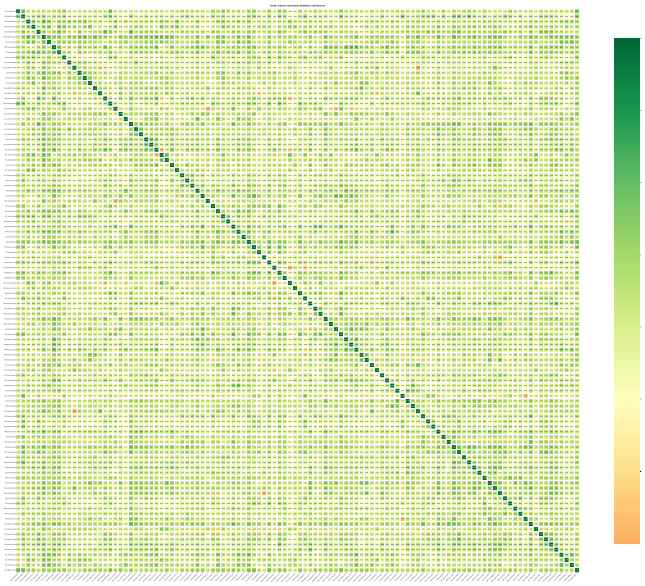


Figure 2: Pairwise Spearman correlation heatmap of the released A-share factor library (110 admitted factors), computed from cross-sectionally standardized realized factor signals over the common time–asset panel. The average off-diagonal absolute correlation is Avg $|\rho|$ = 0.203.

with similar competitive improvements on the other markets (see Table 1).

In terms of redundancy, FactorMiner's selected factor set exhibits moderate pairwise dependence. As summarized by Avg $|\rho|$ in Table 1, the average pairwise absolute correlation is 0.30–0.31 on A-shares and 0.25 on Crypto. The correlation distribution further shows a controlled tail ($|\rho| \approx 0.44$–0.45 on A-shares and 0.42 on Crypto), suggesting that the performance gains are not driven by a few near-duplicate signals. We further visualize the correlation structure of the full admitted factor library in Figure 2. The complete factor formulas are provided in Section D.

4.2.2 Robustness Across Heterogeneous Markets. The cross-market evaluation reveals the generalization capability of the discovered factors. While the Cryptocurrency market represents a fundamentally different microstructure (24/7 trading, no price limits) and carries its own distinct risk-factor structure compared to A-shares [19], FactorMiner maintains competitive performance on Crypto, with IC/ICIR of 3.82%/0.28 in the single-factor library evaluation and 9.48%/0.62 under a simple IC-weighted combination (see Table 1).

This robustness suggests that FactorMiner captures fundamental price-volume dynamics (e.g., liquidity-constrained reversals, volatility clustering) that are invariant across asset classes, rather than overfitting to the idiosyncrasies of a single market.

To explain why a single admission threshold need not fit all markets, we characterize each market by cross-sectional dispersion (CSD) [6], the within-bar return dispersion across assets, and return synchronicity (SYN) [22], the degree to which assets co-move with the market aggregate (Table 2). A-share universes exhibit larger cross-sectional dispersion, whereas Crypto shows positive

Table 1: Out-of-Sample performance comparison with a stricter protocol. Top-40 factors are selected on CSI500 (2024) and evaluated on 2025 across datasets. For Alpha101, Classic is restricted to the same candidate set as Adapted. All reported IC and ICIR use the paper's absolute-IC summary: $|\mathbb{E}[IC_t]|$ and $|\mathbb{E}[IC_t]|/\mathrm{std}(IC_t)$. Factor Combination uses the frozen Top-40 with EW and ICW (weights/signs determined on 2024). Factor Selection trains on 2024 and tests on 2025 using Lasso and XGBoost.

Dataset	Method	Factor Library (Top-40)			Factor Combination (Top-40)				Factor Selection (Train'24/Test'25)			
		IC (%)	ICIR	Avg $\lvert\rho\rvert$	EW IC	EW ICIR	ICW IC	ICW ICIR	Las. IC	Las. ICIR	XGB. IC	XGB. ICIR
CSI500	RF‡	2.68	0.25	**0.13**	6.98	0.41	12.02	0.90	13.81	1.15	8.99	0.90
	Alpha101 (Classic)	4.49	0.42	0.19	10.85	0.88	12.89	0.99	14.09	**1.27**	12.07	1.21
	Alpha101 (Adapted)	5.06	0.43	0.21	11.53	0.86	14.71	1.13	13.70	1.08	13.76	1.20
	GPLearn	6.04	0.43	0.44	10.30	0.62	13.38	1.00	12.44	1.17	9.86	0.95
	AlphaForge	4.48	0.38	0.36	7.12	0.60	11.13	0.93	10.49	0.87	11.30	1.25
	AlphaAgent	5.90	0.46	0.32	10.99	0.92	11.86	0.99	13.87	1.19	11.93	1.24
	FactorMiner (Ours)	**8.25**	**0.77**	0.31	**14.95**	**1.29**	**15.11**	**1.31**	**14.59**	1.21	**14.03**	**1.29**
CSI1000	RF‡	2.88	0.30	**0.13**	7.48	0.49	12.28	1.00	13.72	1.24	9.54	1.03
	Alpha101 (Classic)	4.86	0.50	0.19	11.37	1.02	13.14	1.08	**14.64**	**1.42**	11.11	1.17
	Alpha101 (Adapted)	5.32	0.49	0.21	11.95	0.98	**14.78**	1.21	13.08	1.09	**13.88**	1.26
	GPLearn	5.86	0.48	0.44	11.10	0.73	13.66	1.11	12.87	1.23	9.53	0.96
	AlphaForge	4.64	0.42	0.35	7.60	0.71	11.25	1.02	10.42	0.93	12.20	**1.34**
	AlphaAgent	6.21	0.51	0.32	11.17	1.04	12.00	1.12	13.73	1.27	11.42	1.22
	FactorMiner (Ours)	**7.78**	**0.76**	0.30	**14.62**	**1.37**	14.76	**1.39**	14.25	1.25	12.42	1.30
CSI300	RF‡	1.94	0.15	**0.13**	5.46	0.30	9.49	0.61	9.98	0.63	5.55	0.46
	Alpha101 (Classic)	3.44	0.26	0.18	8.55	0.57	10.31	0.65	11.84	0.85	9.70	0.74
	Alpha101 (Adapted)	4.00	0.28	0.20	9.31	0.60	12.03	0.76	12.04	0.77	**11.81**	0.90
	GPLearn	4.12	0.16	0.45	8.01	0.44	10.64	0.66	9.59	0.58	7.92	0.61
	AlphaForge	3.53	0.25	0.36	5.19	0.35	8.54	0.57	6.79	0.43	10.89	0.91
	AlphaAgent	4.69	0.30	0.33	8.83	0.59	9.65	0.65	**12.26**	**0.86**	9.66	0.84
	FactorMiner (Ours)	**7.46**	**0.38**	0.31	**12.49**	**0.88**	**12.66**	**0.88**	11.23	0.82	11.33	**0.94**
Crypto	RF‡	1.45	0.09	**0.07**	3.31	0.19	7.91	0.48	8.04	0.51	3.50	0.24
	Alpha101 (Classic)	2.11	0.14	0.14	5.91	0.41	8.05	0.53	**9.63**	**0.54**	5.61	0.37
	Alpha101 (Adapted)	2.40	0.15	0.15	6.09	0.41	9.21	**0.62**	8.45	0.48	4.51	0.30
	GPLearn	2.50	0.15	0.38	4.44	0.25	7.62	0.42	9.07	0.48	4.04	0.27
	AlphaForge	2.52	0.16	0.31	4.63	0.29	7.44	0.44	8.39	0.42	5.00	0.33
	AlphaAgent	2.86	0.17	0.27	7.94	0.34	8.40	0.36	7.48	0.41	2.35	0.17
	FactorMiner (Ours)	**3.82**	**0.28**	0.25	**9.48**	**0.61**	**9.48**	**0.62**	8.98	0.51	**6.82**	**0.47**

Note: IC is computed as Spearman rank correlation per bar and summarized as $|\mathbb{E}[IC_t]|$. Las.=Lasso, XGB.=XGBoost. ‡RF=Random Formula Exploration.

SYN, indicating stronger market-wide co-movement. This heterogeneity motivates adaptive admission: in lower-dispersion, higher-synchronicity markets such as Crypto, a lower cross-sectional IC threshold and timing-oriented objectives are more informative, while the richer cross-sectional dispersion of A-shares makes strict cross-sectional admission more meaningful.

Table 2: Market heterogeneity measured by cross-sectional dispersion (CSD, ‰) and return synchronicity (SYN) over half-year windows.

Metric	Period	CSI500	CSI1000	CSI300	Crypto
CSD (‰)	2024H1	5.58	5.00	3.50	2.74
	2024H2	5.05	10.56	6.44	2.64
	2025H1	3.87	4.73	3.09	2.39
	2025H2	4.26	4.41	3.64	2.48
	Avg	4.69	6.18	4.17	2.56
SYN	2024H1	-1.04	-0.80	-1.41	0.47
	2024H2	-0.41	-0.49	-0.48	0.47
	2025H1	-1.31	-1.28	-1.40	0.80
	2025H2	-2.01	-2.01	-2.14	0.90
	Avg	-1.19	-1.14	-1.36	0.66

4.2.3 Ensembles vs. Learned Selection. Beyond individual factors, we evaluate the utility of the mined libraries for downstream portfolio construction via two families of methods: simple factor combinations (equal-weight and IC-weighted) and learned selection models (Lasso and XGBoost trained on 2024 and evaluated on 2025). Across most baselines, learned models provide a clear uplift over simple ensembles, consistent with the presence of heterogeneous yet partially redundant signals in the candidate libraries. For FactorMiner, however, learned selection offers limited additional gains and can be slightly negative on some datasets (e.g., CSI500: EW and ICW ICIR = 1.29/1.31 vs. Lasso and XGBoost = 1.21/1.29), indicating that simple ensemble signals already capture most of the exploitable predictive power under the Train'24/Test'25 protocol (see Table 1).

4.2.4 Additional Robustness Checks. We further compare against end-to-end forecasting baselines that directly map the 10-minute market panel to predictive scores: PatchTST [25], Chronos-2 [3], and LightGBM [15]. These neural or tree-based predictors provide nontrivial signals, but they do not match the accuracy and stability of the interpretable formulaic library. As shown in Table 3, FactorMiner achieves stronger IC/ICIR than these end-to-end baselines across the evaluated markets. The gap is largest on A-shares, where LightGBM reaches 5.53%/0.51 on CSI500 while FactorMiner reaches 8.25%/0.77.

The end-to-end signals are nevertheless complementary: their average absolute correlation with the released 110-factor library is below 0.04 and the maximum single-factor correlation is at most 0.23, while pairwise correlations among the three outputs stay modest (Section B). Neural predictors may thus offer useful auxiliary information for future hybrid ensembles, while FactorMiner retains transparent, auditable formulas.

Table 3: Comparison with end-to-end forecasting baselines on the 2025 test period. IC is in %; higher is better.

Market	PatchTST		Chronos-2		LightGBM		FactorMiner	
	IC (%)	ICIR	IC (%)	ICIR	IC (%)	ICIR	IC (%)	ICIR
CSI500	2.21	0.19	4.52	0.37	5.53	0.51	**8.25**	**0.77**
CSI1000	1.84	0.20	3.36	0.26	4.85	0.45	**7.78**	**0.76**
CSI300	1.80	0.15	3.01	0.19	4.66	0.34	**7.46**	**0.38**
Crypto	1.58	0.06	2.62	0.16	2.88	0.13	**3.82**	**0.28**

To test temporal persistence beyond the main 2025 evaluation window, we also evaluate the full 110-factor library on CSI500 over recent months through 2026-Q1. Table 4 shows that both the raw library and XGBoost-selected composite signal remain predictive in 2026-01 to 2026-03, supporting the view that the mined factors generalize beyond the original training year rather than merely fitting 2024-specific patterns. These results highlight a practical trade-off between factor timeliness and long-run stability: markets evolve, published anomalies may erode after becoming known and arbitraged [20], and intraday microstructure signals can have shorter half-lives, making evidence under the current market regime especially important [1].

Table 4: Recent-period robustness of the full factor library on CSI500.

Period	Lib. IC (%)	Lib. ICIR	XGB IC (%)	XGB ICIR
2025-10	5.52	0.49	13.42	1.16
2025-11	6.74	0.65	15.83	1.46
2025-12	6.83	0.68	16.16	1.55
2026-01	4.78	0.46	12.56	1.03
2026-02	6.23	0.64	15.23	1.38
2026-03	5.52	0.51	14.96	1.17

The learned selector is also useful as a diagnostic tool for understanding which parts of the library carry nonlinear marginal value. Table 5 shows that XGBoost importance is not concentrated in a single VWAP-style anchor: the top factors span VWAP deviation, modernized Alpha101 formulas, momentum, regime switches, price-range interactions, divergence, higher moments, median-based transforms, and amount-efficiency signals. This broad support helps explain why tree-based selection improves several weaker libraries, while FactorMiner's own simple ensembles are already competitive because the admitted library is more diverse before learning.

4.3 Effect of Experience Memory

To isolate the contribution of the Experience Memory ($\mathcal{M}$), we conducted an ablation study comparing FactorMiner against a "No Memory" variant where operators F, E, R were disabled, reducing the system to standard LLM-based evolution (see Figure 3 for a summary). Since this experiment is designed as a controlled mechanism ablation, we adopt relatively relaxed screening thresholds to ensure sufficient samples for comparison (IC threshold $|\text{IC}| > 0.02$; redundancy threshold $\theta = 0.85$ for this ablation setting). Figure 3 highlights the impact of memory guidance:

- **Precise Navigation (High Yield)**: The *Have Memory* variant generates 96 high-quality candidates (60.0% yield), whereas the No Memory baseline produces only 32 (20.0% yield). This suggests that the Retrieval operator effectively maps the search space, allowing the agent to guide exploration toward regions with higher expected yield rather than exploring randomly.
- **Aggressive Filtration (High Diversity)**: Despite generating 3.0× more valid signals, the *Have Memory* variant actively rejects a higher proportion of them for redundancy (55.2% vs 43.8%). This confirms that the Evolution operator (E) functions as a strategic filter, prioritizing unique signal discovery over mere quantity.

Table 5: XGBoost feature importance on the full FactorMiner library (top 20 factors).

Rank	ID	Factor	Imp.	Category
1	006	VWAP Deviation	6.04	VWAP
2	061	Alpha101-12 Modern	4.06	Classic
3	023	Norm.-Momentum TsRank	3.59	Momentum
4	068	Skewness Regime PV-Div	3.55	Regime
5	028	Close-Low × Volume	3.03	Price range
6	070	Price-Pos. Vol. Interact.	2.27	Interaction
7	057	TsRank PV Divergence	2.26	Divergence
8	029	High-Close × Volume	2.15	Price range
9	018	Range-Position Vol. Regime	1.62	Range
10	045	Kurtosis-Regime Range	1.57	Higher moment
11	048	Vol-Price Rank Div.	1.47	Divergence
12	104	Median LogSwap	1.46	Median
13	053	Alpha101-1-V	1.44	Classic
14	101	Median Returns Switch	1.41	Median
15	004	High-Vol Rel. Weakness	1.34	Volume
16	055	Skew Open-Close Prod.	1.24	Distribution
17	019	VWAP-Gap Regime Range	1.20	VWAP
18	092	Amt Efficiency EMA	1.16	Efficiency
19	073	Amt Velocity Regime	1.16	Efficiency
20	043	Skewed Volume Momentum	1.11	Distribution

Beyond aggregate counts, the retrieved memories expose reusable search priors rather than opaque prompts. Table 6 summarizes representative directions that repeatedly led to admitted, low redundancy factors. These priors make the search less trial-and-error: the agent is steered toward mechanism families such as higher moment regimes, amount efficiency, and trend regression adaptivity, while later memory updates can suppress directions that collapse back into crowded VWAP deviation clusters.

Table 6: Recommended mining directions distilled from experience memory.

Pattern	Search prior	Yield
Higher moments	Use Skew/Kurt as regime conditions for reversal logic.	High
PV corr. interaction	Combine price-volume correlation with amount or trend operators.	High
Robust efficiency	Smooth return/amount efficiency using median or robust statistics.	High
Smoothed efficiency rank	Apply EMA/WMA before cross-sectional ranking.	High
Trend regression	Switch between Slope and Resi using Rsquare reliability.	High
Extreme regimes	Use logical Or over price/volume extremes as state triggers.	High
Kurtosis regime	Adapt reversal windows under fat-tail environments.	High
Amt-efficiency interaction	Cross amount-efficiency rank with Skew/Kurt features.	Med.

4.4 Mining Efficiency Analysis

To demonstrate the practical advantage of the factor mining skill, we conduct rigorous benchmarks comparing three execution backends

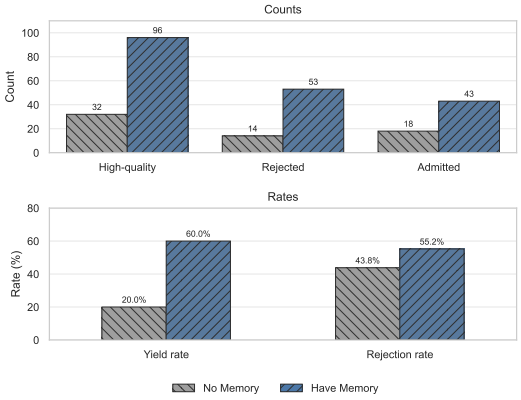


Figure 3: Ablation comparison between *Have Memory* and No Memory. High-quality candidates who pass the IC threshold. The bar chart reports the counts and the corresponding yield and rejection rates.

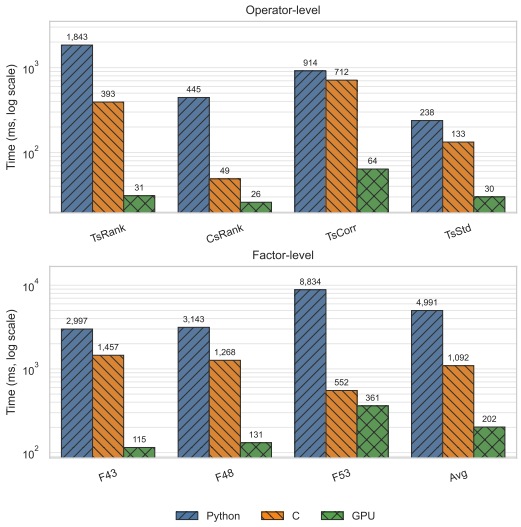


Figure 4: Operator and factor level computation time (ms, log scale) under Python, optimized CPU (C), and GPU backends; lower is better. The GPU backend yields consistent order-of-magnitude gains.

supported by our framework: (1) standard **Python (Pandas)**, (2) **C-compiled (Bottleneck)** for efficient CPU execution, and (3) **GPU-accelerated** for massive parallelism. This multi-backend support allows the agent to remain lightweight while achieving industrial-grade efficiency through on-demand acceleration.

Results. Figure 4 reports both operator-level and factor-level benchmarks on the CSI500 dataset (12,610 × 500), measuring *pure computation time* excluding I/O. At the operator level, the GPU backend achieves **8–59**× speedups over Pandas and **2–13**× over the optimized C backend. The C backend is valuable for lightweight CPU-only runs, while the GPU backend is decisive for repeated large-batch evaluation.

For end-to-end factor evaluation (Figure 4, factor level), rank-intensive factors (F43, F48, F53) show substantial **23–27**× speedups, and even the highly-optimized C implementation is **5.4**× slower than the GPU backend on average (1,092ms vs. 202ms). By offloading these intensive primitives to GPU/C while parallelizing batch evaluation, FactorMiner keeps large-scale iterative discovery feasible on commodity hardware: evaluating 1,000 candidate factors takes ∼6 minutes versus ∼70 minutes with Pandas, keeping the iterative "generate-evaluate-refine" loop practical.

5 Discussion

High-frequency markets can be treated as a data-rich complex adaptive system, and each formulaic factor can be viewed as an explicit, falsifiable hypothesis about market microstructure. Beyond predictive performance, FactorMiner yields 110 interpretable high-frequency alpha factors and a standardized evaluation protocol for reproducible, hypothesis-driven analysis of market microstructure.

Experience memory as continual learning. Our results confirm that memory-guided, skill-based agents can scale neural-symbolic program synthesis while retaining interpretability. The experience memory acts as institutional knowledge accumulated across sessions, enabling meta-learning: the system learns not just individual factors but how to search more effectively. Key insights include **successful patterns** (higher-moment regimes via Skew/Kurt, trend-regression adaptivity via Rsquare/Slope/Resi, amount-efficiency interactions) that yield high-IC low-correlation factors; **failure patterns** (VWAP-deviation variants, standardized returns, simple Delta reversals) that correlate with existing factors; and the **Correlation Red Sea**, where new orthogonal factors get harder to find as the library grows without memory guidance.

6 Conclusion

FactorMiner provides a lightweight self-evolving agent framework for interpretable high-frequency alpha discovery by combining a modular mining skill with experience memory. The resulting 110-factor library and standardized protocol form a reproducible artifact that supports hypothesis testing, mechanistic inspection, and cross-market transfer studies of market microstructure. Future work will incorporate transaction-cost-aware backtesting, extend to broader assets and frequencies, and develop online memory updates for non-stationary markets.

7 Limitations and Ethical Considerations

Although we include comparisons with end-to-end models, these baselines are not exhaustively tuned and remain less interpretable than formulaic factors. Our pipeline also targets predictive IC, ICIR, redundancy, and portfolio diagnostics, with transaction-cost-aware deployment left for future work. The discovered factors could be misused in speculative or manipulative strategies; deployment should follow compliance and risk-control requirements.

Acknowledgments

This work was supported in part by the Shenzhen Ubiquitous Data Enabling Key Lab under Grant ZDSYS20220527171406015, and in part by the Tsinghua Shenzhen International Graduate School-Shenzhen Pengrui Endowed Professorship Scheme of Shenzhen Pengrui Foundation.

References

[1] Yacine Aït-Sahalia, Jianqing Fan, Lirong Xue, and Xiaonan Zhu. 2025. How and When Are High-Frequency Stock Returns Predictable? *Management Science* (2025). doi:10.1287/mnsc.2022.02435 Articles in Advance.

[2] Franklin Allen and Risto Karjalainen. 1999. Using Genetic Algorithms to Find Technical Trading Rules. *Journal of Financial Economics* 51, 2 (1999), 245–271. doi:10.1016/S0304-405X(98)00052-X

[3] Abdul Fatir Ansari, Oleksandr Shchur, Jaris Küken, Andreas Auer, Boran Han, Pedro Mercado, Syama Sundar Rangapuram, Huibin Shen, Lorenzo Stella, Xiyuan Zhang, Mononito Goswami, Shubham Kapoor, Danielle C. Maddix, Pablo Guerron, Tony Hu, Junming Yin, Nick Erickson, Prateek Mutalik Desai, Hao Wang, Huzefa Rangwala, George Karypis, Yuyang Wang, and Michael Bohlke-Schneider. 2025. Chronos-2: From Univariate to Universal Forecasting. *arXiv preprint arXiv:2510.15821* (2025). doi:10.48550/arXiv.2510.15821

[4] Charles Blundell, Benigno Uria, Alexander Pritzel, Yazhe Li, Avraham Ruderman, Joel Z. Leibo, Jack Rae, Daan Wierstra, and Demis Hassabis. 2016. Model-Free Episodic Control. *arXiv preprint arXiv:1606.04460* (2016). doi:10.48550/arXiv.1606.04460

[5] Eugene F. Fama and Kenneth R. French. 1993. Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics* 33, 1 (1993), 3–56. doi:10.1016/0304-405X(93)90023-5

[6] Tianlun Fei, Xiaoquan Liu, and Conghua Wen. 2019. Cross-sectional return dispersion and volatility prediction. *Pacific-Basin Finance Journal* 58 (2019), 101218. doi:10.1016/j.pacfin.2019.101218

[7] Guanhao Feng, Stefano Giglio, and Dacheng Xiu. 2020. Taming the Factor Zoo: A Test of New Factors. *The Journal of Finance* 75, 3 (2020), 1327–1370. doi:10.1111/jofi.12883

[8] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*. PMLR, 1126–1135. https://proceedings.mlr.press/v70/finn17a.html

[9] Shihao Gu, Bryan Kelly, and Dacheng Xiu. 2020. Empirical Asset Pricing via Machine Learning. *The Review of Financial Studies* 33, 5 (2020), 2223–2273. doi:10.1093/rfs/hhaa009

[10] Abhishek Gupta, Russell Mendonca, YuXuan Liu, Pieter Abbeel, and Sergey Levine. 2018. Meta-Reinforcement Learning of Structured Exploration Strategies. *arXiv preprint arXiv:1802.07245* (2018). doi:10.48550/arXiv.1802.07245 NeurIPS 2018.

[11] Campbell R. Harvey and Yan Liu. 2021. Lucky Factors. *Journal of Financial Economics* 141, 2 (2021), 413–435. doi:10.1016/j.jfineco.2021.04.014

[12] Campbell R. Harvey, Yan Liu, and Heqing Zhu. 2016. … and the Cross-Section of Expected Returns. *The Review of Financial Studies* 29, 1 (2016), 5–68. doi:10.1093/rfs/hhv059

[13] Zura Kakushadze. 2016. 101 Formulaic Alphas. *Wilmott* 2016, 84 (2016), 72–81. doi:10.1002/wilm.10525 arXiv:1601.00991.

[14] Ehud Karpas, Omri Abend, Yonatan Belinkov, Barak Lenz, Opher Lieber, Nir Ratner, et al. 2022. MRKL Systems: A Modular, Neuro-Symbolic Architecture that Combines Large Language Models, External Knowledge Sources and Discrete Reasoning. *arXiv preprint arXiv:2205.00445* (2022). doi:10.48550/arXiv.2205.00445

[15] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*, Vol. 30. 3146–3154. https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[16] John R. Koza. 1992. *Genetic Programming: On the Programming of Computers by Means of Natural Selection.* MIT Press, Cambridge, MA.

[17] Serhiy Kozak, Stefan Nagel, and Shrihari Santosh. 2020. Shrinking the Cross-Section. *Journal of Financial Economics* 135, 2 (2020), 271–292. doi:10.1016/j.jfineco.2019.06.008

[18] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 3102–3116. doi:10.18653/v1/2023.emnlp-main.187

[19] Yukun Liu, Aleh Tsyvinski, and Xi Wu. 2022. Common Risk Factors in Cryptocurrency. *The Journal of Finance* 77, 2 (2022), 1133–1177. doi:10.1111/jofi.13119

[20] R. David McLean and Jeffrey Pontiff. 2016. Does Academic Research Destroy Stock Return Predictability? *The Journal of Finance* 71, 1 (2016), 5–32. doi:10.1111/jofi.12365

[21] Alberto Moraglio, Krzysztof Krawiec, and Colin G. Johnson. 2012. Geometric Semantic Genetic Programming. In *Parallel Problem Solving from Nature – PPSN XII*. Lecture Notes in Computer Science, Vol. 7491. Springer, 21–31. doi:10.1007/978-3-642-32937-1_3

[22] Randall Morck, Bernard Yeung, and Wayne Yu. 2000. The information content of stock markets: why do emerging markets have synchronous stock price movements? *Journal of Financial Economics* 58, 1-2 (2000), 215–260. doi:10.1016/S0304-405X(00)00071-4

[23] Christopher J. Neely, Paul A. Weller, and Rob Dittmar. 1997. Is Technical Analysis in the Foreign Exchange Market Profitable? A Genetic Programming Approach. *Journal of Financial and Quantitative Analysis* 32, 4 (1997), 405–426. doi:10.2307/2331231 JSTOR:2331231.

[24] Alex Nichol, Joshua Achiam, and John Schulman. 2018. On First-Order Meta-Learning Algorithms. *arXiv preprint arXiv:1803.02999* (2018). doi:10.48550/arXiv.1803.02999

[25] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*. https://openreview.net/forum?id=Jbdc0vTOcol

[26] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2024). doi:10.48550/arXiv.2310.08560

[27] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual Lifelong Learning with Neural Networks: A Review. *Neural Networks* 113 (2019), 54–71. doi:10.1016/j.neunet.2019.01.012

[28] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*. 1–22. doi:10.1145/3586183.3606763

[29] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334* (2023). doi:10.48550/arXiv.2305.15334 NeurIPS 2024.

[30] Riccardo Poli, William B. Langdon, and Nicholas F. McPhee. 2008. *A Field Guide to Genetic Programming*. Lulu.com. http://www.gp-field-guide.org.uk/

[31] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. *arXiv preprint arXiv:2307.16789* (2023). doi:10.48550/arXiv.2307.16789 ICLR 2024.

[32] Cynthia Rudin. 2019. Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead. *Nature Machine Intelligence* 1, 5 (2019), 206–215. doi:10.1038/s42256-019-0048-x

[33] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761* (2023). doi:10.48550/arXiv.2302.04761 NeurIPS 2023.

[34] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. *arXiv preprint arXiv:2303.17580* (2023). doi:10.48550/arXiv.2303.17580 NeurIPS 2023.

[35] Hao Shi, Weili Song, Xinting Zhang, Jiahe Shi, Cuicui Luo, Xiang Ao, Hamid Arian, and Luis Seco. 2025. AlphaForge: A Framework to Mine and Dynamically Combine Formulaic Alpha Factors. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 12 (2025), 12524–12532. doi:10.1609/aaai.v39i12.33365

[36] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366* (2023). doi:10.48550/arXiv.2303.11366 NeurIPS 2023.

[37] Trevor Stephens and contributors. [n. d.]. gplearn: Genetic Programming in Python. GitHub repository. https://github.com/trevorstephens/gplearn Accessed 2026-02-08.

[38] Ziyi Tang, Zechuan Chen, Jiarui Yang, Jiayao Mai, Yongsen Zheng, Keze Wang, Jinrui Chen, and Liang Lin. 2025. AlphaAgent: LLM-Driven Alpha Mining with Regularized Exploration to Counteract Alpha Decay. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2813–2822. doi:10.1145/3711896.3736838

[39] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023). doi:10.48550/arXiv.2305.16291 Transactions on Machine Learning Research (TMLR), 2024.

[40] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629* (2022). doi:10.48550/arXiv.2210.03629 ICLR 2023.

[41] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. 2023. Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 5476–5486. doi:10.1145/3580305.3599831

[42] Junjie Zhao, Chengxi Zhang, Min Qin, and Peng Yang. 2025. QuantFactor REINFORCE: Mining Steady Formulaic Alpha Factors with Variance-Bounded REINFORCE. *IEEE Transactions on Signal Processing* 73 (2025), 2448–2463. doi:10.1109/TSP.2025.3576781

A Operator Library and Admission Criteria

Each symbolic factor $\alpha \in \mathcal{P}$ is an expression tree built from raw feature references (`$open`, `$high`, `$low`, `$close`, `$volume`, `$amt`, `$vwap`, `$returns`) and a curated library of 60+ typed operators, organized into the categories shown in Table 7; Table 7 lists representative operators for readability, while the full FactorMiner registry contains 60+ typed operators.

Table 7: Operator categories in the factor mining skill (representative operators; 60+ in total).

Category	Representative operators
Arithmetic	Add, Sub, Mul, Div, Neg, Abs, Log, SignedPower, Power, Inv, Sqrt, Square, Exp, Tanh
Statistical	Mean, Std, Var, Skew, Kurt, Med, Sum, Product
Time-series	Delay, Delta, TsRank, TsMax, TsMin, TsArgMax, TsArgMin, TsDecay
Cross-sectional	CsRank, Scale
Smoothing	SMA, EMA, WMA
Regression	Slope, Rsquare, Resi
Logical	IfElse, Greater, Less, GreaterEqual, LessEqual, And, Or, Eq, Ne

Admission and replacement. A symbolic factor α is admitted to the library $\mathcal{L}$ if it satisfies

$$\mathrm{IC}(\alpha) \geq \tau_{\mathrm{IC}} \quad \wedge \quad \max_{g \in \mathcal{L}} |\rho(\alpha, g)| < \theta, \tag{10}$$

where τ_{IC} and θ are the IC and correlation thresholds (defaults $\tau_{\mathrm{IC}} = 0.04$ and $\theta = 0.5$ for A-share mining unless otherwise specified). We further introduce a factor replacement mechanism for high-quality candidates that would otherwise be rejected due to correlation: if a new factor α satisfies

$$\mathrm{IC}(\alpha) \geq 0.10 \ \wedge \ \mathrm{IC}(\alpha) \geq 1.3\,\mathrm{IC}(g) \ \wedge \ |\{g \in \mathcal{L} : \rho(\alpha, g) > \theta\}| = 1, \tag{11}$$

then α replaces the single correlated factor g, allowing the library to evolve toward higher quality while preserving diversity.

Reproducibility. Symbolic proposals are generated by Gemini 3.0 Flash without any weight updates. Factor evaluation uses GPU/C-accelerated operators (NumPy/CuPy) and a 40-worker multiprocessing pool; representative operator runtimes are reported in Figure 4. Alpha101 baselines, originally daily-frequency, are adapted to 10-minute bars by generating up to 10 window-parameter variants per formula and keeping the best configuration.

B End-to-End Signal Correlations

We test whether the end-to-end baselines (PatchTST, Chronos-2, LightGBM) are redundant with the released 110-factor library. Table 8 shows the overlap is small: average absolute correlations are below 0.04 and the maximum single-factor correlation is at most 0.23. Table 9 shows that pairwise correlations among the three signals are also modest. These models do not dominate the formulaic library in predictive performance, but their outputs may carry complementary information for future hybrid systems.

C Experience Memory: Forbidden Directions

Beyond successful templates, the memory also distills *negative knowledge*: structural directions that repeatedly produce candidates highly correlated with the existing library. Table 10 summarizes the main forbidden directions and their typical correlation with admitted factors; recording them prevents the agent from wasting budget rediscovering known signals.

Table 8: Correlation between end-to-end model signals and the 110-factor library.

Model	Avg $\lvert\rho\rvert$	Max $\lvert\rho\rvert$	Nearest factor	2nd	3rd
PatchTST	0.0365	0.18	002 EMA Deviation	108	006
Chronos-2	0.0399	0.23	062 Amount-stability adj. returns	002	081
LightGBM	0.0146	0.08	062 Amount-stability adj. returns	006	084

Table 9: Pairwise correlations among end-to-end signals.

Model	Chronos-2	LightGBM	PatchTST
Chronos-2	1.000	0.043	0.171
LightGBM	0.043	1.000	0.224
PatchTST	0.171	0.224	1.000

Table 10: Forbidden mining directions (high correlation risk).

Direction	Correlated factors	$\lvert\rho\rvert$
Standardized Returns/Amount	006, 008, 009	>0.6
VWAP Deviation variants	006, 009, 012, 013, 016	>0.5
Mean Reversion / Price Deviation	001, 002	>0.5
Simple Delta Reversal	023, 024	>0.5
Close-Position Location	028, 044	0.87+
Volatility + Price Position Branch	046, 064	>0.6
High R^2 Trend Following	081, 083	>0.6
Rsquare Weighted Momentum	002, 023	>0.7
WMA/EMA Smoothed Efficiency	092	>0.9

The VWAP cluster (factor 006 and variants) is the densest correlation region: close-to-VWAP signals normalized by volatility or volume almost always correlate >0.5 with it. As the library grew past 70 factors, even high-IC candidates increasingly collided with existing ones, indicating the "easy" signal space was largely exhausted.

D Full Factor Library (110 factors)

ID	Formula
001	`Neg(CsRank(Div(Sub($close, Min($close, 48)), Add(Sub(Max($close, 48), Min($close, 48)), 1e-8))))`
002	`Neg(Div(Sub($close, EMA($close, 10)), EMA($close, 10)))`
003	`Sub(CsRank(Delta($volume, 1)), CsRank(Div(Sub($close, $vwap), $vwap)))`
004	`Mul(CsRank(Div($volume, Mean($volume, 24))), CsRank(Neg($returns)))`
005	`Neg(Mul(TsRank(Div(Sub($close, TsMin($close, 12)), Add(Sub(TsMax($close, 12), TsMin($close, 12)), 1e-6)), 12), CsRank(Abs(Cov($returns, $volume, 12)))))`
006	`Neg(Div(Sub($close, $vwap), $vwap))`
007	`Neg(Sub(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001))), CsRank(Delta(EMA($volume, 5), 5))))`
008	`Neg(Mul(CsRank(Div($returns, Add(Std($returns, 12), 0.0001))), CsRank(Div($volume, Mean($volume, 12)))))`
009	`Neg(Mul(CsRank($returns), CsRank(Std($returns, 12))))`
010	`Neg(CsRank(Sub(CsRank(Div($close, Delay($close, 5))), CsRank(Div($volume, Delay($volume, 5))))))`
011	`Neg(Sub(CsRank(Delta($close, 6)), CsRank(Delta($vwap, 6))))`
012	`Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), CsRank(Div($volume, EMA($volume, 12))))`
013	`Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), Sub(1, CsRank($volume)))`
014	`Mul(Add(Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), 0.6), Mul(CsRank(Delta(Neg(Div(Sub($close, $vwap), $vwap)), 3)), 0.4)), CsRank(Delta(Div(Mean(Div(Abs($returns), Add($volume, 1)), 24), Add(Div(Abs($returns), Add($volume, 1)), 1e-6)), 3)))`

```
Mul(CsRank(Neg(Delta($returns, 3))), Mul(Sub(1, CsRank(Std($returns, 12))), CsRank(Std($returns, 12))))
Neg(Mul(CsRank(Delta(Div(Sub($close, $vwap), $vwap), 3)), CsRank(Div($volume, EMA($volume, 12)))))
Neg(Mul(TsRank(Delta(Div(Sub($close, $vwap), $vwap), 3), 12), TsRank(Std($returns, 12), 12)))
Neg(Mul(TsRank(Div(Sub($close, TsMin($close, 12)), Add(Sub(TsMax($close, 12), TsMin($close, 12)), 1e-6)), 12), TsRank(Div($volume, EMA($volume, 12)), 12)))
IfElse(Greater(Abs(Div(Sub($close, $vwap), $vwap)), 0.01), Neg(CsRank(Sub($close, TsMax($close, 12)))), Neg(CsRank(Sub($close, TsMin($close, 12)))))
TsRank(Sub(CsRank(Delta($volume, 3)), CsRank(Delta($close, 3))), 18)
Neg(Mul(TsRank(Delta(Div($volume, EMA($volume, 12)), 3), 12), TsRank(Delta(Div(Sub($close, $vwap), $vwap), 3), 12)))
Neg(CsRank(Div(Sub(Min2($open, $close), $low), Add(Sub($high, $low), 0.0001))))
Neg(TsRank(Div(Delta($close, 3), Mean(Abs(Delta($close, 3)), 12)), 12))
Sub(TsRank(Delta($open, 6), 12), TsRank(Delta($close, 6), 12))
Sub(TsRank(Delta($open, 9), 18), TsRank(Delta($close, 9), 18))
Neg(Mul(TsRank(Cov(TsRank($close, 18), TsRank($volume, 18), 10), 18), TsRank(Div(Sub($close, $low), Sub($high, $low)), 18)))
Neg(Mul(TsRank(Cov(TsRank($open, 12), TsRank($volume, 12), 5), 12), TsRank(Div(Sub($close, $low), Sub($high, $low)), 12)))
Neg(Mul(Sub($close, $low), Div($volume, Mean($volume, 12))))
Mul(Sub($high, $close), Div($volume, Mean($volume, 12)))
Mul(Delta(Sub($high, $close), 3), Div($volume, Mean($volume, 12)))
IfElse(Greater(Delta($close, 1), 0), Neg(TsRank(Delta($close, 3), 12)), TsRank(Delta($close, 3), 12))
Neg(TsRank(Sub(CsRank(Delta($returns, 2)), CsRank(Delta($volume, 2))), 12))
Neg(Mul(TsRank(Corr(TsRank(Div($volume, Mean($volume, 24)), 12), TsRank(Sub($high, $low), 12), 12), 24), CsRank(Delta($close, 2))))
IfElse(Greater(Delta($close, 2), 0), Neg(TsRank(Delta($close, 5), 15)), TsRank(Delta($close, 5), 15))
Neg(Mul(TsRank(Delta(Sub($high, $low), 2), 12), CsRank(Delta($close, 2))))
IfElse(Greater(Std($returns, 12), Mean(Std($returns, 12), 48)), Neg(CsRank(Div(Sub(Min2($open, $close), $low), Add(Sub($high, $low), 0.0001)))), Neg(CsRank($returns)))
Neg(TsRank(Mul(CsRank(Delta($close, 3)), CsRank(Delta($volume, 3))), 18))
IfElse(Greater(Std($returns, 12), Mean(Std($returns, 12), 48)), Neg(TsRank(Sub(CsRank(Delta($volume, 3)), CsRank(Delta($close, 3))), 18)), Neg(TsRank(Div(Sub($close, $vwap), Add(Std($returns, 24), 0.0001)), 12)))
Neg(Mul(CsRank(Delta($close, 5)), CsRank(Kurt($returns, 24))))
Neg(Mul(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001))), CsRank(Skew($returns, 24))))
Mul(CsRank(Div(Sub($high, $close), Add(Sub($high, $low), 1e-8))), CsRank(Neg(Skew($returns, 24))))
IfElse(Greater(Abs(Skew($returns, 24)), 1.0), Neg(CsRank($returns)), Neg(CsRank(Skew($returns, 24))))
Neg(TsRank(Mul(CsRank(Skew($volume, 24)), CsRank(Delta($close, 3))), 12))
Neg(Mul(CsRank(Kurt($volume, 24)), CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001)))))
IfElse(Greater(Kurt($returns, 24), 3.0), Neg(CsRank(Div(Sub($high, $close), Add(Sub($high, $low), 0.0001)))), Neg(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001)))))
IfElse(Greater(Std($returns, 12), Mean(Std($returns, 12), 48)), Neg(CsRank(Delta($close, 3))), Neg(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001)))))
Neg(Mul(CsRank(TsRank(Delta(Log(Add($volume, 1)), 3), 6)), CsRank(Div(Delta($close, 6), $close))))
TsRank(Sub(CsRank(Std($returns, 12)), CsRank(Delta($close, 6))), 18)
IfElse(Greater(Skew($returns, 24), 0.5), CsRank(Delta($volume, 3)), Neg(CsRank(Std($returns, 12))))
IfElse(Greater(Skew($returns, 24), 0.5), Neg(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 1e-6)))), Neg(CsRank(Delta($close, 6))))
IfElse(Less(Skew($returns, 24), -0.5), CsRank(Delta($close, 5)), Neg(CsRank(Std($returns, 12))))
IfElse(Less(Skew($returns, 24), -0.5), Neg(TsRank($returns, 12)), Neg(CsRank(Delta($close, 6))))
Neg(CsRank(TsArgMax(SignedPower(IfElse(Less($returns, 0), Std($returns, 20), $close), 2), 5)))
IfElse(Greater($amt, Mean($amt, 24)), CsRank(Delta($close, 3)), Neg(CsRank(Div(Sub($close, $vwap), $vwap))))
IfElse(Greater(Corr($close, $volume, 12), 0.5), Neg(CsRank($returns)), CsRank(Delta($volume, 3)))
IfElse(Greater(Kurt($returns, 24), 3.0), Neg(CsRank(Div($amt, Add($volume, 1e-6)))), Neg(CsRank(Delta($close, 3))))
Neg(Sub(TsRank(Delta($close, 6), 24), TsRank(Delta($volume, 6), 24)))
IfElse(Greater($amt, Mean($amt, 48)), Neg(CsRank(Div($amt, Add($volume, 1e-6)))), Neg(TsRank($returns, 12)))
IfElse(Greater(Std($returns, 12), Mean(Std($returns, 12), 24)), Neg(CsRank(Skew($amt, 12))), Neg(CsRank(Delta($close, 3))))
IfElse(Greater(Div($amt, Mean($amt, 24)), 1.2), Sub(CsRank($close), CsRank($volume)), Neg(CsRank(Delta($close, 3))))
Neg(Mul(TsRank(Delta($volume, 1), 12), TsRank(Delta($close, 1), 12)))
Neg(Div(CsRank($returns), Add(Std($amt, 12), 1e-6)))
Neg(TsRank(Add(Add(CsRank(Delta($returns, 3)), CsRank(Delta($amt, 3))), CsRank(Kurt($volume, 12))), 12))
IfElse(Less(Corr($close, $volume, 12), -0.5), CsRank($returns), Neg(CsRank(Delta($close, 3))))
IfElse(Greater(Std($returns, 12), Mean(Std($returns, 12), 48)), Neg(CsRank(Delta($amt, 6))), Neg(CsRank(Delta($close, 1))))
IfElse(Greater(Kurt($volume, 24), 3), Neg(CsRank(Delta($amt, 3))), Neg(CsRank($returns)))
Neg(Mul(CsRank(Div(Sub($low, $close), Add(Sub($low, $high), 1e-6))), CsRank(Delta($amt, 6))))
Neg(IfElse(Less(Skew($returns, 24), -0.5), Neg(Sub(TsRank($close, 12), TsRank($volume, 12))), CsRank(Delta($returns, 3))))
Neg(IfElse(Greater(Kurt($returns, 12), 3.0), Neg(CsRank(Std($returns, 6))), CsRank(Delta($returns, 3))))
Neg(Mul(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 0.0001))), TsRank(Std($returns, 24), 24)))
Mul(CsRank(Div(Sub($high, $close), Add(Sub($high, $low), 1e-6))), TsRank(Std($volume, 12), 12))
IfElse(Greater(Kurt($volume, 24), 3.0), Neg(Corr($close, $volume, 12)), Neg(TsRank($returns, 24)))
IfElse(Greater($amt, EMA($amt, 12)), Neg(TsRank(Delta($volume, 3), 12)), Neg(CsRank($returns)))
Neg(Mul(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 1e-6))), TsRank(Skew($volume, 24), 12)))
Neg(Mul(CsRank(Div($returns, $amt)), TsRank(Std($volume, 20), 20)))
Neg(IfElse(Greater(Div($amt, Mean($amt, 24)), 2.0), Neg(Delta($close, 1)), SMA($returns, 6)))
Neg(Mul(TsRank(Std(Div($volume, Mean($volume, 24)), 24), 24), TsRank($returns, 12)))
Neg(Mul(CsRank(Div($returns, $amt)), TsRank(Kurt($volume, 20), 20)))
IfElse(Greater(Std($returns, 12), EMA(Std($returns, 12), 48)), Neg(Div(Sub($close, $low), Add(Sub($high, $low), 1e-6))), Div(Sub($high, $close), Add(Sub($high, $low), 1e-6)))
IfElse(Greater(Rsquare($close, 24), 0.7), Neg(CsRank(Slope($close, 24))), Neg(CsRank(Resi($close, 12))))
IfElse(And(Greater(Rsquare($close, 24), 0.6), Greater(Std($returns, 12), Mean(Std($returns, 12), 48))), Slope($close, 12), Neg(SMA($returns, 6)))
IfElse(Or(Greater(Div(Sub($high, Max2($open, $close)), Add(Sub($high, $low), 1e-6)), 0.6), Greater(Div(Sub(Min2($open, $close), $low), Add(Sub($high, $low), 1e-6)), 0.6)), Slope($close, 12), Neg(SMA($returns, 6)))
IfElse(And(Greater(Rsquare($close, 24), 0.5), Less(Std($returns, 12), Mean(Std($returns, 12), 48))), Slope($close, 12), Neg($returns))
IfElse(Eq(Sign(Delta(Std($returns, 12), 1)), Sign(Delta($returns, 1))), Neg(Resi($close, 12)), Neg(Slope($close, 24)))
Neg(Mul(CsRank(Rsquare($close, 24)), CsRank(Resi($close, 12))))
IfElse(Eq(Sign(Delta(Resi($close, 12), 1)), Sign(Resi($close, 12))), Neg(Resi($close, 6)), Neg(Slope($close, 12)))
IfElse(Greater(Kurt($returns, 24), 3.0), Neg(Resi($close, 6)), Neg(Resi($close, 24)))
Neg(Mul(CsRank(Div($returns, Add($amt, 1e-6))), TsRank(Skew($returns, 24), 24)))
IfElse(Or(Greater($volume, Mean($volume, 24)), Greater(Abs($returns), Std($returns, 24))), Neg(Resi($close, 6)), Neg(Slope($close, 12)))
IfElse(Greater(Rsquare($close, 24), 0.75), Neg(Slope($close, 12)), Neg(TsRank($returns, 12)))
Neg(Mul(TsRank(Div($returns, Add($amt, 1e-6)), 24), TsRank(Kurt($returns, 24), 24)))
Neg(CsRank(EMA(Div($returns, Add($amt, 1e-6)), 6)))
Neg(TsRank(Delta(Div($returns, Add($amt, 1e-6)), 3), 24))
IfElse(And(Greater(Std($returns, 12), Mean(Std($returns, 12), 60)), Less(Kurt($returns, 24), 2)), Neg(Delta($close, 3)), Neg(TsRank($returns, 24)))
IfElse(Or(Greater(Abs(Skew($returns, 24)), 1.5), Greater(Kurt($returns, 24), 4.0)), Neg(Resi($close, 6)), Neg(TsRank($returns, 24)))
Neg(Mul(TsRank(WMA(Div($returns, Add($amt, 1e-6)), 6), 24), TsRank(Corr($close, $volume, 24), 24)))
Neg(CsRank(Med(Div($returns, Add($amt, 1e-6)), 6)))
Neg(CsRank(Delta(WMA(Div($returns, Add($amt, 1e-6)), 6), 3)))
Neg(Sub(EMA(Div($returns, Add($amt, 1e-6)), 6), EMA(Div($returns, Add($amt, 1e-6)), 24)))
Neg(Resi(Delta(Resi($close, 24), 3), 12))
Neg(IfElse(Greater(Abs(Skew($returns, 24)), 1.0), TsRank(Div(Sub($returns, Med($returns, 24)), Add(Std($returns, 24), 1e-6)), 24), CsRank(Resi($close, 12))))
IfElse(Greater(Skew($high, 24), 0), TsRank(Log(Div($high, Add($close, 1e-6))), 24), CsRank(Resi($low, 24)))
Neg(IfElse(Greater(Skew($open, 24), 0), TsRank(Resi($open, 24), 24), CsRank(SignedPower($returns, 0.5))))
Neg(IfElse(Greater(Med($returns, 24), 0), TsRank(Log(Div($close, Add($open, 1e-6))), 24), CsRank(Sign(Resi($close, 24)))))
Neg(IfElse(Greater(Skew($returns,24),0.4), CsRank(Sign(Resi($open,24))), TsRank(SignedPower($returns,0.6),24)))
Neg(IfElse(Greater(Std($volume,24),Mean(Std($volume,24),48)), CsRank(Div($amt, Add($volume,1e-6))), TsRank(Div($returns, Add(Std($returns,24),1e-6)),24)))
IfElse(Greater(Std($returns, 12), Med(Std($returns, 12), 48)), Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), CsRank(Div($volume, EMA($volume, 12)))), Neg(TsRank(Div(Sub($close, $vwap), $vwap), 24)))
Neg(IfElse(Less(Skew($returns, 24), -0.3), Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), CsRank(Div($volume, EMA($volume, 12)))), TsRank(Div(Sub($close, $vwap), $vwap), 12)))
IfElse(Greater(Rsquare($close, 24), 0.6), Neg(CsRank(Slope($close, 24))), Mul(CsRank(Neg(Div(Sub($close, $vwap), $vwap))), CsRank(Div($volume, EMA($volume, 12)))))
Neg(Mul(CsRank(Div(Sub($close, $low), Add(Sub($high, $low), 1e-6))), CsRank(Mul(Skew($returns, 24), Slope($close, 12)))))
```